

Kawakiri: an open-source automated framework for dimensional schema inference from undocumented databases

Nour el houda Ramdane¹, Maxime Berger¹, Mickaël Martin Nevot¹, and Anthony Tinson²

¹ KOUSHIN, Marseille, France ² Aix-Marseille School of Economics (AMSE) CNRS UMR 7316, Aix-Marseille Université, Marseille, France

DOI: 10.xxxxxx/draft

Software

- [Review](#)
- [Repository](#)
- [Archive](#)

Editor: [↗](#)

Submitted: 02 July 2026

Published: unpublished

License

Authors of papers retain copyright and release the work under a Creative Commons Attribution 4.0 International License ([CC BY 4.0](#)).

Summary

Kawakiri is an open-source platform that extracts decision-making models and reconstructs candidate dimensional deterministic models from various undocumented sources. Using column-based database profiling, functional dependencies, key and join inference, graph topology analysis, and explicit validation rules, Kawakiri generates auditable snowflake, star, or constellation models.

When given a folder containing undocumented tabular data sources, such as raw CSV exports, Kawakiri ingests the data into ClickHouse. It then profiles every column, reconstructs non-overlapping functional column groups, materializes logical tables, infers candidate primary keys and join relationships, classifies logical tables into fact and dimension roles, assembles candidate dimensional-model graphs (snowflake, star or constellation), and validates each candidate against a fixed set of structural rules. Finally, it certifies each candidate in a JSON report. Unlike other tools, which require analysts to declare keys and relationships upfront, Kawakiri derives these elements from statistical evidence in the data itself, such as uniqueness, entropy, null ratios, and aggregation behavior. It only accepts a candidate model once it has passed referential, granularity, semantic, and aggregation-stability checks. The output is a validated, auditable dimensional model, along with generated SQL views. This is intended to shorten the path from an undocumented data export to a query-ready analytical schema.

Statement of need

Traditionally, designing multidimensional schemas for business intelligence has relied on a comprehensive understanding of source systems. Although automated and semi-automated multidimensional design approaches have been widely studied ([Romero & Abelló, 2009](#)), they often assume clean, well-documented source schemas or require significant expert intervention.

Teams that specialize in analytics and data engineering often receive tabular extracts, such as CSV dumps from legacy systems, data lake exports, and third-party feeds. These extracts typically lack documentation of primary or foreign keys, as well as the intended fact/dimension roles. Manually building a dimensional model from these sources is time-consuming, and the results are often ambiguous; several relationship graphs can fit the same noisy source tables equally well. Manual modeling does not address this non-identifiability problem systematically.

Kawakiri is designed for data and analytics engineers who need to create a preliminary dimensional model from undocumented relational extracts before finalizing a production warehouse design. It is also intended for students and professionals studying automated

approaches to dimensional modeling. However, Kawakiri is not a substitute for domain expertise. It produces a certified candidate model with an explicit, inspectable trail of evidence showing which statistical tests were passed and which structural rules were checked. With this information, a human modeler can confirm, reject, or refine the model rather than receiving a black-box recommendation.

State of the field

There are two broad categories of existing tooling for dimensional-model construction.

Declarative transformation and testing frameworks such as dbt (dbt Labs, 2026) and Dataform (Google Cloud, 2026), let engineers encode SQL transformations and assertions, such as uniqueness and referential integrity tests, once the model's keys and relationships are known. These frameworks do not attempt to discover relationships from the data itself. Therefore, an incorrect or incomplete manual declaration of keys can propagate silently into the rest of the pipeline.

Traditional reverse engineering and profiling tools such as SchemaSpy (SchemaSpy Contributors, 2026), reconstruct relationship diagrams from physical database metadata, such as declared FOREIGN KEY constraints and naming conventions. However, these tools provide little value for denormalized CSV or data lake sources, where no such metadata exists. Data-quality frameworks, such as Great Expectations (Great Expectations Contributors, 2026), compute column-level statistics, such as completeness and cardinality, but do not link these statistics to dimensional modeling hypotheses or structural validation rules, such as deterministic granularity and aggregation stability. These are both well-established concerns in dimensional modeling methodology (Kimball & Ross, 2013).

Kawakiri's contribution is combining the two by inferring keys, joins, and fact/dimension roles statistically without requiring declared constraints. Although column names can provide weak secondary evidence, physical and statistical tests ultimately make the decision. Kawakiri then applies the same class of structural checks to the resulting candidate model that a dimensional-modeling practitioner would apply manually: referential integrity, granularity, semantic separation, and aggregation stability. In research and teaching contexts involving undocumented or rapidly changing source schemas, this approach eliminates the implicit assumption in dbt, Dataform, and similar tools that the dimensional structure is already known. It also goes beyond what general-purpose profilers report by tying statistical evidence directly to an accept/reject decision on the candidate model.

Software design

Rather than a single monolithic inference step, Kawakiri is organized as a sequential, inspectable pipeline:

- > ingestion
- > profiling
- > identifiability scoring
- > preliminary source-key and relationship inference
- > functional-dependency grouping
- > logical fact/dimension materialization
- > logical-table profiling
- > primary-key inference
- > join inference
- > adjacency-graph construction
- > fact/dimension role inference
- > candidate-model construction

- > ranking
- > structural validation
- > granularity validation
- > semantic validation
- > aggregation-stability validation
- > certification
- > (optional) SQL view generation
- > JSON report export

Each stage saves its output to a metadata store kept in a database that is separate from the analytical data being modeled (META_DB vs. CH_DATABASE), so the candidate model, its supporting statistics, and its validation issues can be examined separately. Individual CLI stages can be rerun as long as their prerequisite metadata is still available. The run-all command deliberately rebuilds computed metadata to ensure a complete, reproducible execution.

ClickHouse was chosen as the underlying engine because the profiling stage involves columnar aggregations, such as distinct-value counts, null ratios, and entropy estimates, over potentially large tables. ClickHouse (Schulze et al., 2024) is optimized for this access pattern, and the same engine hosts the inferred model's generated SQL views, which avoids a second runtime dependency.

Since the noise tolerance appropriate for a given source system is domain-dependent, join-acceptance and identifiability thresholds (e.g., the join-success-ratio cutoff θ_{jst}) are exposed as configuration rather than hard-coded. Validation rules are implemented as independent, named checks rather than a single pass/fail function so that a certification report can specify which rule a candidate failed and the reason why.

Methodology

Data profiling and functional dependencies

Kawakiri closes the gap with rule-based database reverse engineering. It uses column-based data profiling to automatically infer functional and inclusion dependencies. Instead of relying on non-deterministic heuristic searches, Kawakiri uses an axiom-based synthesis workflow grounded in established relational data-profiling and dependency-discovery methods (Abedjan et al., 2015).

Mathematics and algorithms

Kawakiri does not infer keys, joins, or table roles based solely on column names; rather, each hypothesis is verified using physical or statistical evidence derived from the data. Before key inference, functional dependencies are used to reconstruct coherent logical groups. A column is only attached to a group after passing a dependency test, while unassigned columns remain explicit singletons in the grouping metadata.

Metric	Interpretation
uniqueness_ratio	Distinct values over total rows; close to 1 supports an identifier
null_ratio	Share of missing values; a key candidate should be close to 0
entropy_ratio	Normalized Shannon entropy; high values support identifier-like columns
identifiability_score	Combined uniqueness, entropy, and completeness score used to rank primary-key candidates

Metric	Interpretation
variation_coefficient	Normalized numeric variability; high values can indicate a fact measure

Identifiability and semantic classification. For a column C with distinct values $X = \{x_1, \dots, x_n\}$ and empirical probabilities $P(x_i)$, the engine computes the Shannon entropy $H(C) = -\sum_i P(x_i) \log_2 P(x_i)$ (Shannon, 1948), normalized as $H_{norm}(C) = H(C) / \log_2(N)$, where N is the row count. Since $H_{norm}(C) \rightarrow 1$ only when nearly every value is unique ($n \rightarrow N$), this normalization is deliberately stricter than the conventional entropy normalization by $\log_2(n)$: it is designed to identify columns that act as identifiers rather than to measure category diversity in general. Low-entropy columns support a dimension classification, and high-entropy, high-variability numeric columns support a fact classification. Coupled with the coefficient of variation $CV(C) = \sigma/\mu$ for numeric columns (undefined when $\mu = 0$, in which case the column is excluded from variability-based scoring), this classification occurs.

Join inference and topology. A directed edge from a candidate foreign key C_s in table T_s to a candidate primary key K_t in table T_t is accepted when the join success ratio

$$JSR(C_s \rightarrow K_t) = \frac{N_{matched}}{N_{source, non-null}}$$

meets a configurable tolerance θ_{jsr} (the default is 0.95) and where $N_{matched}$ is the number of rows successfully joined rows, and $N_{source, non-null}$ is the number of non-null rows in the source column.

The accepted edges form an adjacency matrix, A , which is checked for cycles via a depth-first traversal. A candidate model is rejected if A is not acyclic, since a cyclic join graph cannot be resolved into a well-defined star, snowflake, or constellation shape.

Aggregation stability. For a measure M in fact table F and a categorical attribute G of dimension D , the engine compares the row-level sum $\Sigma_{fine} = \sum_{i \in F} M_i$ to the sum obtained after a left join and a GROUP BY on G , Σ_{agg} . A model is accepted only if $|\Sigma_{fine} - \Sigma_{agg}| \leq \epsilon$. This check assumes a one-to-one join on the tested key. Therefore, a legitimate multi-valued dimension attribute must be excluded or pre-aggregated before this test, which is a known limitation documented alongside the validator.

Validation rules

Currently, five structural rules gate certification:

- **Referential integrity:** fact-to-dimension links should not create orphan values.
- **Topology:** the join graph must be acyclic, with no self-loops or invalid fact-to-fact links.
- **Deterministic granularity:** fact rows must be uniquely identified by the grain induced by their dimension keys.
- **Semantic separation:** dimension tables should not behave like fact tables, and fact tables should not be dominated by descriptive attributes.
- **Aggregation stability:** measures must remain stable when aggregated through dimensional levels (see above).

A candidate model's final status — VALID, WARNING, or INVALID — is determined by combining its ranking with the outcome of these five rules.

Research impact statement

The Kawakiri method was developed and validated through an end-to-end integration test suite. This suite exercises the full pipeline against a reference dataset with a known star schema ground truth (one fact table, four dimensions, and one deliberately isolated table). The suite asserts that the table-role classification, join inference, and final certification status match the expected model. Each pipeline stage and validation rule is implemented and tested as an independent unit. The certification report format is designed so that new structural rules can be added without altering the report schema.

Future work

The final candidate schemas, generated by Kawakiri using star, snowflake, or constellation models, structurally reconstruct the implicit analytical hierarchies embedded within the raw tables. This algorithmic synthesis provides an automated, concrete foundation for advanced analytical frameworks that formalize hierarchical data cubes and multidimensional lattices (Martin Nevot et al., 2023), ensuring that the inferred candidates respect logical constraints before being materialized.

AI usage disclosure

Generative AI assistance was used during the development of this project, specifically OpenAI Codex, Anthropic Claude 3.5 Sonnet, and GitHub Copilot: (1) to draft and debug portions of the pipeline source code, (2) to draft portions of the project documentation, and (3) to draft and revise this paper, including its structure and compliance with JOSS submission requirements. All AI-assisted code and text were reviewed and validated by the authors before being merged, and no AI-generated contribution was accepted without this review process. The authors made the core design decisions, including the staged pipeline architecture, the choice of statistical metrics, the set of structural validation rules, and the database separation between analytical and metadata storage.

Acknowledgements

The authors would like to express their sincere gratitude to Naël Turlure for his invaluable advice and assistance; to Chiheb Bradai, the technical project manager at Koushin, for his guidance; and to the members of the Aix-Marseille School of Economics laboratory, as well as to others at Koushin Salaries, for their thoughtful counsel. This work did not receive any dedicated financial support.

References

- Abedjan, Z., Golab, L., & Naumann, F. (2015). Profiling relational data: A survey. *The VLDB Journal*, 24(4), 557–581. <https://doi.org/10.1007/s00778-015-0389-y>
- dbt Labs. (2026). *dbt: Data build tool*. Online documentation. <https://docs.getdbt.com>
- Google Cloud. (2026). *Dataform: SQL-based data transformation*. Online documentation. <https://cloud.google.com/dataform>
- Great Expectations Contributors. (2026). *Great Expectations: A python data quality and validation framework*. GitHub repository (Archived on Zenodo). <https://doi.org/10.5281/zenodo.18224886>

- 180 Kimball, R., & Ross, M. (2013). *The data warehouse toolkit: The definitive guide to*
181 *dimensional modeling* (Third edition). John Wiley & Sons, Inc. ISBN: 978-1-118-53080-1
- 182 Martin Nevot, M., Nedjar, S., & Lakhal, L. (2023). Hierarchical Datacubes. *Journal of*
183 *Computer and Communications*, 11(06), 43–72. <https://doi.org/10.4236/jcc.2023.116004>
- 184 Romero, O., & Abelló, A. (2009). A Survey of Multidimensional Modeling Methodologies:
185 *International Journal of Data Warehousing and Mining*, 5(2), 1–23. <https://doi.org/10.4018/jdwm.2009040101>
- 186
- 187 SchemaSpy Contributors. (2026). *SchemaSpy: Database schema documentation and analysis*
188 *tool*. GitHub repository. <https://github.com/schemaspy/schemaspy>
- 189 Schulze, R., Schreiber, T., Yatsishin, I., Dahimene, R., & Milovidov, A. (2024). ClickHouse
190 - Lightning Fast Analytics for Everyone. *Proceedings of the VLDB Endowment*, 17(12),
191 3731–3744. <https://doi.org/10.14778/3685800.3685802>
- 192 Shannon, C. E. (1948). A Mathematical Theory of Communication. *Bell System Technical*
193 *Journal*, 27(3), 379–423. <https://doi.org/10.1002/j.1538-7305.1948.tb01338.x>

DRAFT